



SE2020 – Web Technologies and Mobile

Lab Activity 10: Node Js

Introduction

Node.js is a server side JavaScript runtime environment designed for building scalable and high-performance applications using an event-driven, non-blocking I/O model. This practical session provides hands on experience with Node.js fundamentals through the development of a **console-based Library Management System**.

Intended Learning Outcomes

At the end of this practical session, students will be able to:

1. Explain the role of Node.js as a server-side runtime environment.
2. Apply JavaScript objects and arrays to model real-world data.
3. Implement event-driven logic using functions and user input.
4. Use Node.js core modules such as `fs` and `readline`.
5. Implement file-based data persistence using JSON.
6. Develop a menu-driven console application using Node.js.

Lab Scenario

You are required to design and implement a **Library Management System** using Node.js. The system should allow users to:

- Add, search, borrow, return, and delete books
- Display available and borrowed books
- Persist library data using the file system

The application must be **menu driven** and operate via the command line.

Software Requirements

- Node.js (LTS version recommended)
- Code editor (VS Code recommended)
- Command Line / Terminal

Project Setup

1. Create a new project folder

```
LibraryManagementSystem
```

2. Initialize the Node.js project

```
npm init -y
```

3. Create the main application file

```
library.js
```

Task 1. Create a Book Object

Objective: Understand object structure and properties.

Each book should contain: id, title, author, year, isAvailable

```
// Create a single book
let book = {
  id: 1,
  title: " "Harry Potter ",
  author: " J.K. Rowling ",
  year:1997,
  isAvailable: true
};
console.log("Book Title:", book.title);
console.log("Author:", book.author);
console.log("Available:", book.isAvailable);
```

Task 2. Manage Multiple Books Using Arrays

Objective: Store multiple book objects in an array and access them using indices.

```
javascript
let library = [
  {
    id: 1,
    title: "Harry Potter",
    author: "J.K. Rowling",
    year: 1997,
    isAvailable: true
  },
  {
    id: 2,
    title: "The Hobbit",
    author: "J.R.R. Tolkien",
    year: 1937,
    isAvailable: true
  },
];
console.log("Total books:", library.length);
console.log("\nFirst book:", library[0].title);
console.log("Second book:", library[1].title);
```

Task 3: Display All Books

Objective: Use loops to display book details and availability status.

```
let library = [
  { id: 1, title: "Harry Potter", author: "J.K. Rowling", year: 1997, isAvailable: true },
  { id: 2, title: "The Hobbit", author: "J.R.R. Tolkien", year: 1937, isAvailable: true },
];

function displayAllBooks() {
  console.log("\n=== Library Books ===");
  for (let i = 0; i < library.length; i++) {
    let book = library[i];
    let status = book.isAvailable ? "Available" : "Borrowed";
    console.log(`${book.id}.  "${book.title}"  by  ${book.author}  (${book.year})  -  ${status}`);
  }
}

displayAllBooks();
```

Task 4: Add New Books

Objective: Create reusable functions to add books dynamically to the library.

```
function addBook(title, author, year) {  
  let newBook = {  
    id: library.length + 1,  
    title: title,  
    author: author,  
    year: year,  
    isAvailable: true  
  };  
  library.push(newBook);  
  console.log(`\n✓ Added: "${title}" by ${author}`);  
}
```

Task 5: Search Books by Title

Objective: Implement case-insensitive searching using string methods.

```
function searchBook(title) {  
  let found = null;  
  for (let i = 0; i < library.length; i++) {  
    if (library[i].title.toLowerCase() === title.toLowerCase()) {  
      found = library[i];  
      break;  
    }  
  }  
  if (found) {  
    console.log(`${found.title} - ${found.isAvailable ? "Available" : "Borrowed"}`);  
  } else {  
    console.log("Book not found");  
  }  
}
```

Task 6: Borrow and Return Books

Objective: Update book availability using conditional logic.

```
let library = [  
  { id: 1, title: "Harry Potter", isAvailable: true },  
  { id: 2, title: "The Hobbit", isAvailable: true },  
  { id: 3, title: "1984", isAvailable: false }  
];
```

```
function borrowBook(bookId) {
  const book = library.find(b => b.id === bookId);
  if (!book || !book.isAvailable) return;
  book.isAvailable = false;
}

function returnBook(bookId) {
  const book = library.find(b => b.id === bookId);
  if (!book || book.isAvailable) return;
  book.isAvailable = true;
}
```

Task 7: Filter Books

Objective: Display available and borrowed books using array methods such as `filter()`.

```
function showAvailableBooks() {
  library
    .filter(b => b.isAvailable)
    .forEach(b => console.log(b.title));
}

function showBorrowedBooks() {
  library
    .filter(b => !b.isAvailable)
    .forEach(b => console.log(b.title));
}
```

Task 8: Delete a Book

Objective: Remove a book using its unique identifier.

```
function deleteBook(bookId) {
  const index = library.findIndex(b => b.id === bookId);
  if (index === -1) return;
  library.splice(index, 1);
}
```

